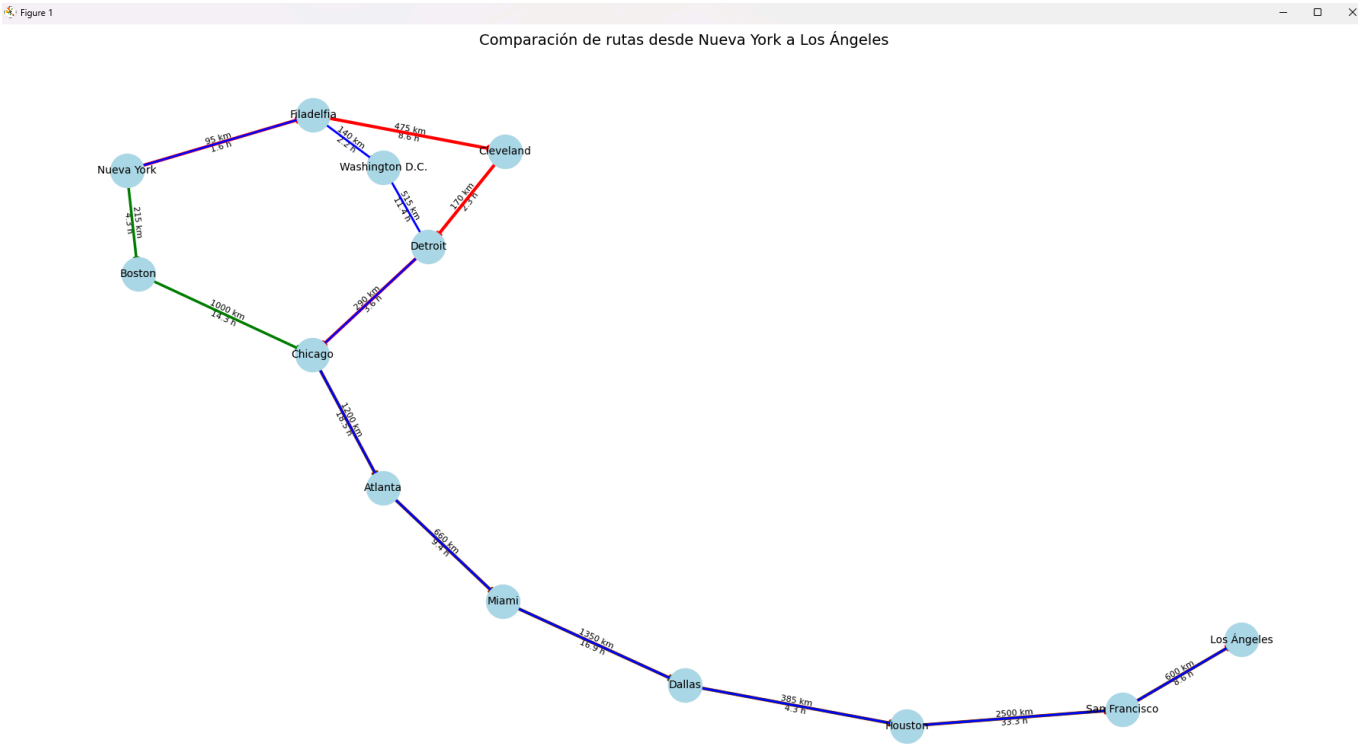


Actividad No. 18

Ejercicio No. 1

Grafo



Ruta Mas Corta

- Ruta más rápida: Nueva York -> Filadelfia -> Cleveland -> Detroit -> Chicago -> Atlanta -> Miami -> Dallas -> Houston -> San Francisco -> Los Ángeles
Tiempo total estimado: 107.06 horas

Rutas Alternativas

```
py main.py
• Ruta más rápida: Nueva York -> Filadelfia -> Cleveland -> Detroit -> Chicago -> Atlanta -> Miami -> Dallas -> Houston -> San Francisco -> Los Ángeles
Tiempo total estimado: 107.06 horas

Ruta alternativa 1: Nueva York -> Boston -> Chicago -> Atlanta -> Miami -> Dallas -> Houston -> San Francisco -> Los Ángeles
Tiempo total estimado: 109.53 horas

Ruta alternativa 2: Nueva York -> Filadelfia -> Washington D.C. -> Detroit -> Chicago -> Atlanta -> Miami -> Dallas -> Houston -> San Francisco -> Los Ángeles
Tiempo total estimado: 109.75 horas
```

Código Python

```

import networkx as nx
import matplotlib.pyplot as plt
import heapq

rutas = {
    ('A', 'C'): (95, 60),
    ('A', 'D'): (215, 50),
    ('C', 'B'): (140, 65),
    ('B', 'F'): (515, 45),
    ('D', 'E'): (1000, 70),
    ('E', 'F'): (290, 80),
    ('C', 'G'): (475, 55),
    ('F', 'G'): (170, 75),
    ('E', 'H'): (1200, 65),
    ('H', 'I'): (660, 70),
    ('I', 'J'): (1350, 80),
    ('J', 'K'): (385, 90),
    ('K', 'L'): (2500, 75),
    ('L', 'M'): (600, 70),
}

nombres_ciudades = {
    'A': 'Nueva York', 'B': 'Washington D.C.', 'C': 'Filadelfia', 'D':
    'Boston',
    'E': 'Chicago', 'F': 'Detroit', 'G': 'Cleveland', 'H': 'Atlanta',
    'I': 'Miami', 'J': 'Dallas', 'K': 'Houston', 'L': 'San Francisco', 'M':
    'Los Ángeles'
}

# construcción del grafo
G = nx.DiGraph()
for (origen, destino), (distancia, velocidad) in rutas.items():
    tiempo = distancia / velocidad
    G.add_edge(origen, destino, distancia=distancia, velocidad=velocidad,
    tiempo=tiempo)
    G.add_edge(destino, origen, distancia=distancia, velocidad=velocidad,
    tiempo=tiempo)

def dijkstra_k_rutas(G, inicio, fin, k=3):
    rutas = []

```

```

cola = [(0, inicio, [])]
visitados_rutas = set()

while cola and len(rutas) < k * 5: # Explorar más para obtener
alternativas
    tiempo_total, nodo, camino = heapq.heappop(cola)
    camino = camino + [nodo]
    if nodo == fin:
        ruta_id = tuple(camino)
        if ruta_id not in visitados_rutas:
            rutas.append((camino, tiempo_total))
            visitados_rutas.add(ruta_id)
        if len(rutas) >= k:
            break
    for vecino in G.neighbors(nodo):
        if vecino not in camino: # evitar ciclos
            tiempo_viaje = G[nodo][vecino]['tiempo']
            heapq.heappush(cola, (tiempo_total + tiempo_viaje, vecino,
camino))
    return rutas[:k]

rutas_resultado = dijkstra_k_rutas(G, 'A', 'M', k=3)

# mostrar rutas
for idx, (ruta, tiempo) in enumerate(rutas_resultado):
    tipo = "Ruta más rápida" if idx == 0 else f"Ruta alternativa {idx}"
    print(f"{tipo}: {' -> '.join(nombres_ciudades[n] for n in ruta)}")
    print(f"Tiempo total estimado: {tiempo:.2f} horas\n")

pos = nx.spring_layout(G, seed=42)
plt.figure(figsize=(18, 12))
nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=1000)
nx.draw_networkx_labels(G, pos, labels=nombres_ciudades, font_size=10)
nx.draw_networkx_edges(G, pos, edge_color='gray')

labels = {(u, v): f"{G[u][v]['distancia']} km\n{G[u][v]['tiempo']:.1f} h" for
u, v in G.edges()}
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels, font_size=8)

# colores para rutas

```

```

colores = ['red', 'green', 'blue']
anchos = [3, 2.5, 2]

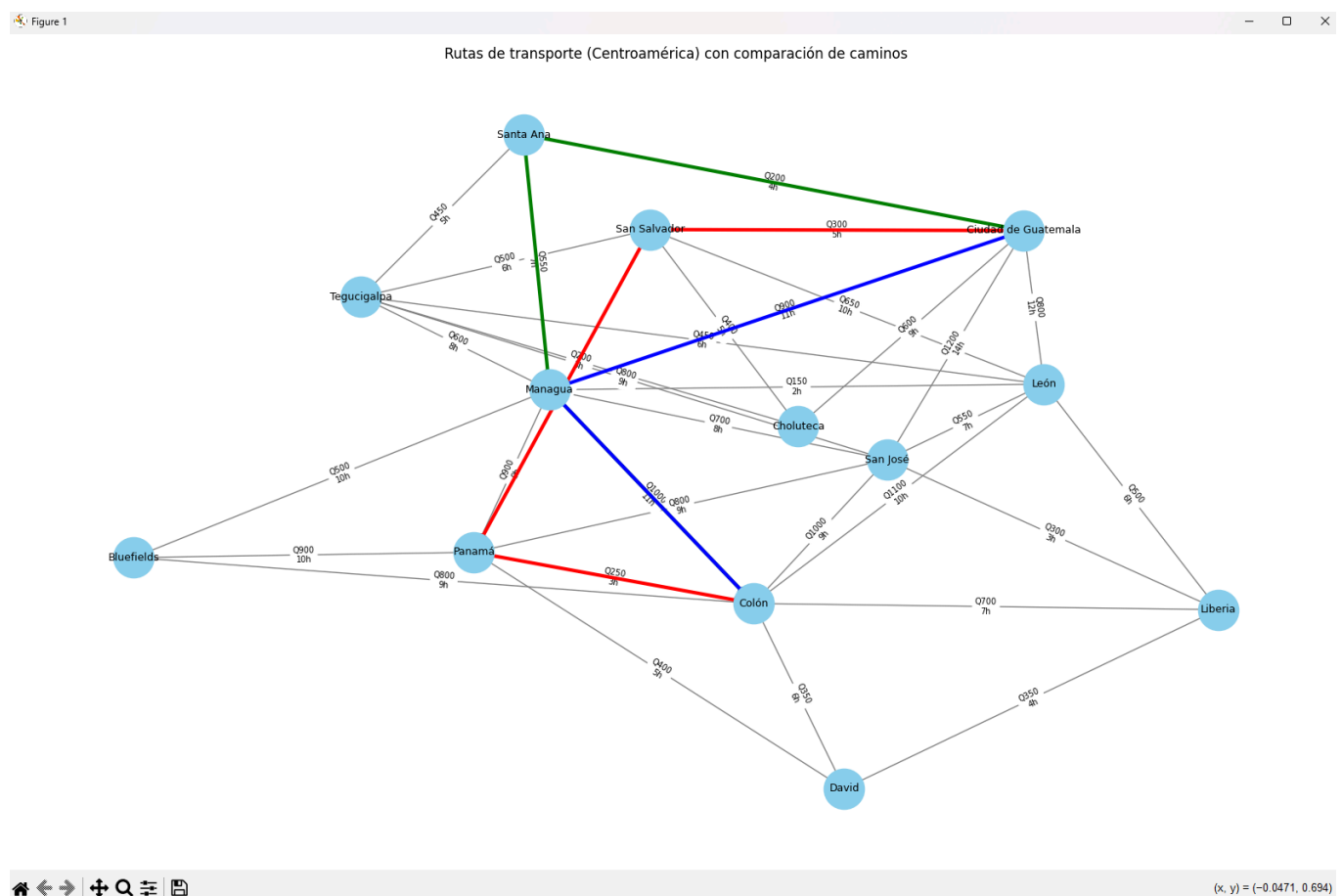
for idx, (ruta, _) in enumerate(rutas_resultado):
    edges = list(zip(ruta[:-1], ruta[1:]))
    nx.draw_networkx_edges(G, pos, edgelist=edges, edge_color=colores[idx],
width=anchos[idx])

plt.title("Comparación de rutas desde Nueva York a Los Ángeles", fontsize=14)
plt.axis('off')
plt.tight_layout()
plt.show()

```

Ejercicio No. 2

Grafo



Ruta más corta

Ruta más económica: Ciudad de Guatemala -> San Salvador -> Panamá -> Colón

Costo total (incluye penalización): Q 1650

Tiempo total de viaje: 20 horas

Penalización por tiempo: Q0

Ruta Alternativa

```
py parser.py

❖ 1. Ruta más económica
  - Ruta: Ciudad de Guatemala -> San Salvador -> Panamá -> Colón
  - Costo total (incluye penalización): Q1650
  - Tiempo total: 20 horas
  - Penalización aplicada: Q0

  2. Ruta alternativa 1
    - Ruta: Ciudad de Guatemala -> Santa Ana -> Managua -> Colón
    - Costo total (incluye penalización): Q1750
    - Tiempo total: 22 horas
    - Penalización aplicada: Q0

  3. Ruta alternativa 2
    - Ruta: Ciudad de Guatemala -> Managua -> Colón
    - Costo total (incluye penalización): Q1900
    - Tiempo total: 22 horas
    - Penalización aplicada: Q0
```

Código Python

```
import networkx as nx
import matplotlib.pyplot as plt
import heapq
import math

ciudades = {
    'A': 'Ciudad de Guatemala',
    'B': 'San Salvador',
    'C': 'Tegucigalpa',
    'D': 'Managua',
    'E': 'San José',
    'F': 'Panamá',
    'G': 'Santa Ana',
    'H': 'Choluteca',
    'I': 'León',
    'J': 'Liberia',
```

```

        'K': 'David',
        'L': 'Colón',
        'M': 'Bluefields'
    }

# Grafo con (destino, costo, horas)
grafo = {

    'A': [('B', 300, 5), ('G', 200, 4), ('H', 600, 9), ('I', 800, 12), ('D',
900, 11), ('E', 1200, 14)],
    'B': [('C', 500, 6), ('I', 650, 10), ('H', 400, 5), ('F', 1100, 12)],
    'G': [('C', 450, 5), ('D', 550, 7)],
    'C': [('D', 600, 8), ('I', 450, 6), ('H', 200, 3), ('E', 800, 9)],
    'D': [('E', 700, 8), ('I', 150, 2), ('M', 500, 10), ('F', 900, 9), ('L',
1000, 11)],
    'I': [('E', 550, 7), ('J', 500, 6), ('L', 1100, 10)],
    'E': [('F', 800, 9), ('J', 300, 3), ('L', 1000, 9)],
    'J': [('K', 350, 4), ('L', 700, 7)],
    'K': [('F', 400, 5), ('L', 350, 6)],
    'F': [('L', 250, 3), ('M', 900, 10)],
    'L': [('M', 800, 9)]
}

```

```

PENALIZACION_DIARIA = 200

```

```

HORAS_DIA = 24

```

```

def dijkstra(grafo, inicio, destino, max_rutas=3):
    cola = [(0, 0, inicio, [])]
    rutas = []
    visitados = set()

    while cola and len(rutas) < max_rutas:
        costo_total, tiempo_total, actual, ruta = heapq.heappop(cola)

        nueva_ruta = ruta + [actual]
        clave_ruta = tuple(nueva_ruta)

        if actual == destino and clave_ruta not in visitados:
            visitados.add(clave_ruta)
            dias_extra = max(0, math.ceil(tiempo_total / HORAS_DIA) - 1)

```

```

        penalizacion = dias_extra * PENALIZACION_DIARIA
        rutas.append({
            'ruta': nueva_ruta,
            'costo_total': costo_total + penalizacion,
            'tiempo_total': tiempo_total,
            'penalizacion': penalizacion
        })

    for vecino, costo, horas in grafo.get(actual, []):
        if vecino not in ruta:
            heapq.heappush(cola, (
                costo_total + costo,
                tiempo_total + horas,
                vecino,
                nueva_ruta
            ))

    return rutas

```

Visualización del grafo

```
def visualizar_grafo(grafo, rutas):
```

```
    G = nx.DiGraph()
```

```
    for origen, conexiones in grafo.items():
```

```
        for destino, costo, horas in conexiones:
```

```
            etiqueta = f"Q{costo}\n{horas}h"
```

```
            G.add_edge(origen, destino, label=etiqueta)
```

```
pos = nx.kamada_kawai_layout(G) # Mejor distribución para claridad
```

```
plt.figure(figsize=(16, 10))
```

```
nx.draw_networkx_nodes(G, pos, node_color='skyblue', node_size=1200)
```

```
nx.draw_networkx_labels(G, pos, labels=ciudades, font_size=9)
```

```
edge_labels = nx.get_edge_attributes(G, 'label')
```

```
nx.draw_networkx_edges(G, pos, edge_color='gray', arrows=True)
```

```
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=7)
```

```
colores = ['red', 'green', 'blue']
```

```
for i, ruta in enumerate(rutas):
```

```
    edges = list(zip(ruta['ruta'], ruta['ruta'][1:]))
```

```
    nx.draw_networkx_edges(G, pos, edgelist=edges, edge_color=colores[i],
```

```
width=3, style='solid')
```

```
plt.title("Rutas de transporte (Centroamérica) con comparación de  
caminos")
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
rutas = dijkstra(grafo, 'A', 'L', max_rutas=3)
```

```
nombres = ["Ruta más económica", "Ruta alternativa 1", "Ruta alternativa 2"]
```

```
for i, r in enumerate(rutas):
```

```
    print(f"\n{i+1}. {nombres[i]}")
```

```
    print("    - Ruta:", " -> ".join([ciudades[n] for n in r['ruta']]))
```

```
    print(f"    - Costo total (incluye penalización): Q{r['costo_total']}")
```

```
    print(f"    - Tiempo total: {r['tiempo_total']} horas")
```

```
    print(f"    - Penalización aplicada: Q{r['penalizacion']}")
```

```
# Visualizar en el grafo
```

```
visualizar_grafo(grafo, rutas)
```